

Техническое задание:

сервис управления задачами



Разработать REST API сервис для управления задачами внутри команд с ролевой моделью, историей изменений, кешированием списка задач и SQL отчетом.

Обязательный стек

- Go
- MySQL
- Redis
- Docker + Docker Compose
- Git
- Swagger/OpenAPI

Сущности и связи

Сущность	Минимальные поля и связи
users	id, email, password_hash, name, created_at
teams	id, name, created_by, created_at; created_by -> users.id
team_members	team_id, user_id, role; уникальная пара team_id + user_id
tasks	id, team_id, title, description, status, created_by, assignee_id, created_at, updated_at, closed_at, version
task_history	id, task_id, changed_by, changes, created_at
task_comments	id, task_id, user_id, content, created_at

Роли и зоны ответственности

Роль	Что может делать
Создатель (owner)	Полный доступ в рамках своей команды: приглашать участников, менять роли, назначать администратора или участника, редактировать любые задачи команды, смотреть аналитику команды.
Администратор (admin)	Может приглашать участников, редактировать любые задачи команды, смотреть аналитику команды. Не может удалить создателя команды или изменить его роль.
Участник (member)	Может видеть свою команду, задачи команды, историю и комментарии. Может создавать задачи. Может редактировать только задачи, где он создатель или исполнитель, с ограничениями ниже.

Правила видимости и доступа

- Команды: пользователь видит только свои команды, где он участник или является создателем.
- Задачи: пользователь не получает задачи чужой команды. GET /tasks требует team_id и проверку участия в команде.
- История и комментарии: доступны только участникам команды этой задачи.
- Аналитика: /teams/{team_id}/stats доступен только создателю или администратору команды и не возвращает данные других команд.
- Исполнитель: должен быть участником той же команды. Назначение пользователя из другой команды запрещено.
- Приглашения и роли: приглашать пользователей может только создатель или администратор. Роль создателя через обычное приглашение не выдается.

Правила редактирования задач

- Создание задачи: любой участник команды может создать задачу в своей команде. Создателем считается текущий пользователь.
- Редактирование любой задачи: доступно создателю или администратору команды.
- Редактирование своей задачи: создатель задачи может менять название, описание, статус и исполнителя, если новый исполнитель состоит в команде.
- Редактирование назначенной задачи: исполнитель может менять статус и оставлять комментарии, но не должен произвольно переназначать задачу.
- Обычный участник не может редактировать задачу, где он не создатель и не исполнитель.
- История: любое изменение задачи должно записывать старые и новые значения, автора изменения и время изменения.

API: пользователи и команды

- POST /api/v1/register: регистрация пользователя.
- POST /api/v1/login: аутентификация, выдача JWT.
- POST /api/v1/teams: создание команды, текущий пользователь становится создателем.
- GET /api/v1/teams: список команд текущего пользователя.
- POST /api/v1/teams/{id}/invite: добавление пользователя в команду, только создатель или администратор.

API: задачи и отчет

- POST /api/v1/tasks: создание задачи в команде.
- GET /api/v1/tasks?team_id=1&status=todo&assignee_id=5&limit=20&offset=0: список задач команды с фильтрами и пагинацией.
- PUT /api/v1/tasks/{id}: обновление задачи с проверкой прав и записью истории.
- GET /api/v1/tasks/{id}/history: история изменений задачи.
- POST /api/v1/tasks/{id}/comments: добавление комментария к задаче.
- GET /api/v1/tasks/{id}/comments: список комментариев задачи.
- GET /api/v1/teams/{team_id}/stats: SQL-отчет по конкретной команде.

SQL отчет

Обязательный отчет: GET /api/v1/teams/{team_id}/stats

Отчет должен возвращать статистику только по одной команде, к которой у пользователя есть доступ создателя или администратора.

- Метрики: задачи по статусам, топ-3 исполнителя по закрытым задачам за 30 дней, среднее время закрытия, количество комментариев по задачам команды.
- SQL: один осмысленный запрос или CTE, без N+1. Нужны JOIN, GROUP BY, агрегаты и работа с датами.
- Доступ: обязательная фильтрация по team_id и проверка прав на команду.

Надежность, кеш

- Транзакции: создание команды и добавление участника выполняются атомарно. Создание или обновление задачи и запись истории выполняются атомарно.
- Конкурентность: конкурентное обновление задачи не приводит к потере изменений. История строится от актуального состояния задачи.
- Redis: кешируется список задач команды с TTL 5 минут. Кеш учитывает команду и фильтры. После изменения задач кеш обновляется или инвалидируется.
- Запуск: Docker Compose для приложения, MySQL и Redis. Конфигурация без хардкода.

Документация

- README с запуском, конфигурацией, миграциями и примерами запросов.
- Swagger/OpenAPI с эндпоинтами, моделями запросов и ответов, ошибками.

Тестирование

- Обязателен лишь один интеграционный тест на SQL отчет.

Что считается выполненным ТЗ

- API покрывает аутентификацию, команды, задачи, комментарии, историю и один эндпоинт аналитики.
- Права доступа реализованы на уровне бизнес-логики. Нет утечек данных между командами.
- Есть транзакционность для создания/обновления задачи и записи истории.
- Есть защита от конкурентных обновлений задач.
- Есть кеширование списка задач в Redis с корректной инвалидацией.
- Есть Swagger/OpenAPI, понятный README и запуск через Docker Compose.

Будет плюсом

- Unit-тесты на бизнес-логику прав доступа
- Ограничение частоты запросов
- Курсорная пагинация
- Структурированные логи
- Circuit breaker